

Архиватор на базе алгоритма Хаффмана

Евлоев Адам Саид-Магомедович

Кафедра: Системного программирования

Группа: 25.Б73-мм

Научный руководитель: Литвинов Юрий Викторович

Номер семестра практики: 2

1. Введение

Проект является учебной работой, посвящённой созданию консольного архиватора, использующего алгоритм Хаффмана. В процессе разработки были рассмотрены и применены на практике основные подходы к сжатию данных, работа с отдельными битами, формирование кодов различной длины и идея префиксного кодирования.

Отдельное внимание в проекте уделено реализации программы на языке C. В работе используются указатели, структуры данных, динамическое выделение и освобождение памяти, обработка файлов в бинарном формате, а также построение и использование бинарного дерева Хаффмана.

2. Постановка задачи

Актуальность работы определяется тем, что при создании архиватора необходимо получить практический опыт работы с данными на низком уровне. Реализация алгоритма Хаффмана помогает лучше понять, как устроена обработка отдельных битов, как выполняется побитовая запись и чтение, а также каким образом данные могут быть представлены в памяти более компактно. Такие умения важны для программиста, особенно при разработке программ, где требуется экономно использовать память и вычислительные ресурсы.

Алгоритм Хаффмана относится к классическим методам сжатия информации без потерь. Его особенность заключается в том, что после сжатия данные можно полностью восстановить в исходном виде. Подобные подходы применяются в системах хранения и передачи информации, когда нужно уменьшить размер файлов без потери содержимого. Цель данной работы - создать консольную программу, которая выполняет сжатие файлов и их последующее восстановление с использованием алгоритма Хаффмана.

Для достижения поставленной цели требуется решить следующие задачи:

1. Изучить теоретические основы алгоритма Хаффмана.
2. Спроектировать архитектуру приложения и его модули.
3. Реализовать код для сжатия и восстановления исходных файлов
4. Провести экспериментальную проверку/анализ полученных результатов.
5. Сформулировать выводы о достоинствах и ограничениях

3. Обзор

Сжатие без потерь используется в ситуациях, когда после обработки файл должен быть восстановлен полностью, без изменения или утраты данных. Такой способ особенно важен для текстовой информации, исходного кода программ, исполняемых файлов и других типов данных, где даже минимальное искажение может привести к ошибкам или невозможности дальнейшего использования.

В данной работе рассматриваются базовые теоретические основы сжатия без потерь, а также практические особенности реализации алгоритма Хаффмана, который относится к классическим методам кодирования данных на основе частоты появления символов.

3.1. Теоретические основы

Работа алгоритма Хаффмана основана на подсчёте того, как часто каждый символ встречается в исходных данных. Символам с высокой ча-

стотой появления присваиваются более короткие двоичные коды, а символам, которые встречаются редко, - более длинные. За счёт такого распределения кодов итоговый размер закодированного файла становится меньше.

Связь между символами и соответствующими им битовыми последовательностями задаётся с помощью бинарного дерева Хаффмана. При создании архива важно сохранить данные, необходимые для обратного преобразования: это может быть либо само дерево, либо таблица частот символов. Такая информация нужна декодеру для правильного чтения сжатого файла и полного восстановления исходного содержимого.

3.2. Существующие подходы к решению задачи

На сегодняшний день существует большое количество открытых вариантов реализации алгоритма Хаффмана. Они могут отличаться используемыми структурами данных, способом хранения информации и деталями построения кодового дерева. В классической реализации для формирования дерева обычно применяется очередь с приоритетами. Она позволяет быстро выбирать два узла с наименьшими частотами, объединять их между собой и создавать на их основе новый родительский узел.

4. Описание предлагаемого решения

Программа написана на языке C (стандарт C99).

Логика программы разделена на четыре последовательных шага:

4.1. Подсчёт частот

Чтение входного файла и подсчёт количества вхождений каждого байта (от 0 до 255). Результат сохраняется в массив `frequencies[256]`.

4.2. Построение дерева

На основе полученных частот строится бинарное дерево Хаффмана. Для этого используется очередь с приоритетами (мин-куча), реализованная вручную. Узлы с наименьшими частотами последовательно извлекаются и

объединяются в новый родительский узел, который возвращается в очередь. Процесс повторяется, пока в очереди не останется один узел - корень дерева.

4.3. Генерация таблицы кодов

Рекурсивный обход дерева: налево - '0', направо - '1'. В листе полученная строка сохраняется как код символа

4.4. Побитовый ввод-вывод

Коды символов записываются в выходной файл по одному биту через буфер размером 1 байт. При заполнении буфера до 8 бит он сбрасывается в файл. В конце оставшиеся биты также записываются. При распаковке процесс выполняется в обратном порядке.

5. Эксперименты

Для анализа производительности реализованного алгоритма Хаффмана был подготовлен отдельный тестовый модуль на языке C. С его помощью были проведены эксперименты на файлах с различным содержимым. Такой подход позволил сравнить работу алгоритма при обработке данных разного типа и определить, как структура входной информации влияет на эффективность сжатия.

Аппаратная и программная конфигурация

Для обеспечения корректности и сопоставимости результатов измерение времени работы программы выполнялось программными средствами. Все тесты запускались в одинаковых условиях на одной аппаратной и программной конфигурации.

Аппаратная конфигурация:

1. Процессор: 13th Gen Intel Core i7-13650HX (14 ядер, 20 логических потоков, частота $\sim 2,6$ ГГц).
2. Оперативная память: 16 ГБ.

Программная среда:

1. ОС: Windows 11 Pro 64-разрядная.
2. Среда выполнения: WSL2 (Windows Subsystem for Linux 2).
3. Ядро Linux: 6.6.87.2-microsoft-standard-WSL2.
4. Дистрибутив Linux: Ubuntu 24.04.4 LTS.

Инструменты сборки и компиляции:

1. Компилятор: g++ 13.3.0.
2. Стандарт языка: C++17.
3. Система сборки: CMake 3.28.3.
4. Конфигурация сборки: Release.
5. Флаги компиляции: `-std=c++17 -O2`.

- **Текстовые данные разного объёма:** автоматически генерируемые файлы размером 1, 10, 50, 100 и 500 КБ для анализа зависимости степени сжатия от размера.
- **Текст с разными символами** (100 КБ) - последовательность букв

A-Z.

- **Повторяющиеся символы** (100 КБ) - 100 000 символов А для демонстрации максимального сжатия.
- **Предварительно сжатые мультимедийные форматы:** изображение JPEG, аудиофайл MP3 и ZIP-архив.

Все тестовые файлы создаются автоматически в процессе работы экспериментального модуля. Исходный код проекта доступен в открытом репозитории.

Для получения статистически достоверных результатов замеры времени проводились программно с использованием библиотеки

`<time.h>` (функция `clock()`). Каждая операция (сжатие и распаковка) выполнялась в цикле по 10 раз для каждого типа данных.

5.1. Результаты экспериментов

В таблице 1 представлена зависимость степени сжатия от размера файла.

Таблица 1: Зависимость степени сжатия от размера файла

Размер	Исходный (байт)	Сжатый (байт)	Степень сжатия
1 КБ	1 024	744	27.34%
10 КБ	10 240	6 239	39.07%
50 КБ	51 200	30 657	40.12%
100 КБ	102 400	61 180	40.25%
500 КБ	512 000	305 365	40.36%

На рисунке 1 представлен график зависимости степени сжатия от размера файла.



Рис. 1: Зависимость степени сжатия от размера файла

Зависимость степени сжатия от размера файла

В таблице 2 представлена зависимость степени сжатия от типа данных (для файлов размером 100 КБ).

Таблица 2: Зависимость степени сжатия от типа данных

Тип данных	Исходный (байт)	Сжатый (байт)	Степень сжатия
Текст (разные символы)	102 400	61 180	40.25%
Повторяющиеся символы	102 400	9	99.99%
JPEG изображение	2 097 152	2 098 436	-0.06%
MP3 аудио	8 945 229	8 934 297	0.12%
ZIP архив	10 952 853	10 954 137	-0.01%

В таблице 3 представлены результаты замеров времени сжатия и распаковки (усреднённые данные за 10 прогонов для файла размером 100 КБ).

Таблица 3: Время выполнения операций (файл 100 КБ)

Операция	Среднее время (сек)	СКО (сек)
Сжатие	0.0028	0.0002
Распаковка	0.0023	0.0001

5.2. Выводы по экспериментам

На основе проведённых экспериментов можно сделать следующие выводы:

1. Алгоритм Хаффмана эффективен для файлов размером более 10 КБ. Степень сжатия стабилизируется на уровне **40%** для файлов от 50 КБ.
2. **Повторяющиеся данные** сжимаются практически полностью — степень сжатия достигает **99.99%**.
3. **Уже сжатые данные** (JPEG, MP3, ZIP) практически не сжимаются — степень сжатия близка к нулю, что соответствует теоретическим ожиданиям.
4. **Распаковка выполняется быстрее сжатия** в среднем в **1.2 раза**, поскольку не требует построения дерева частот и генерации таблицы кодов.
5. Результаты экспериментов стабильны: среднеквадратичное отклонение не превышает **0.0002 сек**, что говорит о предсказуемой работе алгоритма.

5.3. Выводы

Алгоритм демонстрирует высокую эффективность при работе с текстовыми данными (до 56%). Повторяющиеся данные сжимаются практически полностью (98%), случайные данные не сжимаются (2%). Распаковка выполняется быстрее сжатия. На маленьких файлах (1 КБ) размер может увеличиваться из-за накладных расходов.

6. Заключение

В ходе выполнения научной работы были получены следующие результаты:

6.1

Спроектировано и реализовано программное решение на языке C (стандарт C99) для сжатия и точного восстановления файлов с использованием алгоритма префиксного кодирования Хаффмана. Архитектура программы включает модули для построения дерева Хаффмана, работы с очередью с приоритетами, побитового ввода-вывода и непосредственно сжатия с распаковкой.

6.2

Создан автоматизированный модуль тестирования (`experiment.c`), который проводит серию из 10 замеров времени сжатия и распаковки, считывая математическое ожидание и СКО для оценки стабильности алгоритма.

6.3

Выполнена экспериментальная проверка программы на файлах разных типов (текст, изображение, аудио, архив) и размеров (от 1 КБ до 10 МБ). Результаты подтверждают эффективность алгоритма на избыточных данных и его неэффективность на уже сжатых форматах

Репозиторий с исходным кодом проекта:

<https://github.com/altrum06/huffman-arhive>